

# Model

- A model is a pattern, plan, representation (especially in miniature), or description designed to show **the main object or workings** of an object, system, or concept.
- Models allow us to **reason** about things that **do not exist** yet, are intangible, or are not fully understood. A model helps us to understand something. That understanding may allow us **to refine the model or make decisions** based on our expectation of the fidelity and accuracy of the model.

# Example - Models of Universe

- Earth was at the center of the Universe and that everything rotated around it.
- Copernicus' heliocentric model - Newton theory
- Albert Einstein - General Theory of Relativity
- The **simplest model** that gets the **right numbers** should be used.
- Compare this:

$$F_g = \frac{Gm_1m_2}{r^2}$$

$$R_{\mu\nu} - \frac{1}{2}g_{\mu\nu}R + g_{\mu\nu}\Lambda = \frac{8\pi G}{C^4}T_{\mu\nu}$$

# Importance of models I

- Models are a **fundamental** aspect of everything we do in the **EDA industry**.
- Models can **abstract** away the physics that makes transistors operate, they allow us to **verify** a design before it has been built, they allow us to **increase design efficiency** by having tools that can transform one model into another model that is closer to the final implementation.
- What is meaning of **abstract**?
- The semiconductor **development process** is about:
  - creation
  - application
  - transformation of models
- Models are **expensive to create and maintain**, so we must **limit the number of them** that get used in the development process, often resulting in models being used for things that they were never intended for.

# Importance of models II

- If the model contains **too little** information it will result in **inaccurate** results
- Having **too much information is equally bad**. While we may then get very accurate results, the processing time necessary to get that result is not acceptable.
- Engineers are thus continually playing a **trade-off game** between the time and expense of creating models that would enable them to perform a specific task and the value that those models would provide.

# Types of Model I

- There are many types of models:
  - Conceptual
  - Physical
  - Logical
  - Mathematical
  - Statistical
  - Visual...
- A **conceptual model** is a qualitative model that helps to **highlight important connections between processes**. Conceptual models exist primarily in **the problem domain** and not in the solution domain. Conceptual models are often process flow diagrams, Petri Nets, event graphs, or some aspects of the Unified Modeling Language.

# Types of Model II

- A **physical model** is most often something that can be **created, observed, and manipulated**. One or more aspects of the model have characteristics that are **similar to the real system** and thus allow concepts to be analyzed and understood.
- A **mathematical model** enables equations to be analyzed. In our industry most of these will be **numerical models** although occasionally **analytical models** may exist as well. **Statistical models** allow variances to be modeled and to find means and variances.
- **Visual models** help us to **visualize** how something will **operate** within the real world by providing some graphic or visual outputs, often with animations.

# Types of Model III

- A **logical model** includes some aspects of the structure or form of the device that is to be built. This includes **architectural models**, **transaction level models**, and **any model that includes time**. These are thus **implementation models** and help us refine the ideas into a complete solution.

# Models of Computation I

- A model of computation is an attempt to distil the **fundamental aspects of a system's ability to solve problems**.
- A model of computation **abstracts away** many of the details of the machine so that it can be both **fast and accurate** in terms of the analysis which is to be performed. An example of this is the way in which various models of computation treat time.
- There is no one abstraction of a system and thus no one model of computation that is useful for all tasks in a typical ESL flow. **Multiple models of computation** will be required to progress **from an idea to an implementation**.
- The most fundamental model of computation in the **software** world is the **von Neumann architecture**.
- For the **gate and RT** levels of abstraction, the predominant model on which all languages have been based is the **discrete event model**.



# Models of Computation II

- **Languages** are ways to **map** the capabilities of a **model of computation** into something that can be **described and communicated**.
- It is often thought that the **language is the syntax** and the **model of computation** underlying it is the **semantics**.
- There are languages that have been constructed that do **not have a complete model of computation**.
- Languages such as **C** **encode the limitations of von Neumann architecture**.
- It is clear that we need **different models of computation to describe hardware systems and software systems** even though they share many of the same needs, such as a way to express concurrency and time.

# Simplification - Abstraction

- **Models allow us to simplify** a problem, often down to the essential elements necessary to perform a particular function, computation, or type of analysis. The primary means of simplification are **abstraction, elimination, and structure**.
- In the **software** realm, the processor directly implies the **lowest level** of software abstraction, that is, the **instruction**. The processor takes care of everything below the instruction level.
- A language like **C** adds another layer of abstraction, in that each statement written in C will result in several instructions being created when passed through a compiler.
- In the **hardware world**, there are two primary means of abstraction. These are **time and data**.

# Simplification - Structure

- While a final chip is a flat structure, models can represent this in a hierarchy that enables us to group certain things together, organize by function, perform duplication, and **divide a problem**.
- If you tried to describe a system in a **fully flattened** form, very **few tools would be able to handle the size and complexity** of the resulting system. Hierarchy is very necessary and helpful.

# Models and Languages I

- A **modeling language** is any artificial language that can be used to **express a set of concepts** in a consistent way such that **independent tools** can be built upon that language and can in most cases lead to **exactly the same interpretation of the model**.
- Languages are basically built from three sets of concepts, namely **semantics**, **abstract syntax**, and a **concrete syntax**.
- We have already discussed **semantics**, especially in the context of the **models of computation**. The **abstract syntax** is used to describe in a language neutral way how **components are connected structurally**, but says nothing about the semantics of those connections. And finally the language **syntax** itself defines the **keywords**, how are they formed, the detailed rules for **expressions**, etc.

# Models and Languages II

- **Most programming languages** in use today are **imperative languages**. This is a class of languages in which **statements are executed in order and modify the state of the system**. The dominance of imperative languages should come as no surprise because computers themselves are imperative machines.
- While imperative languages describe a set of ordered statements that are to be executed, **declarative languages state what needs to be accomplished**, but not the way in which it will get done. Emerging use for declarative languages comes from the formal side of the verification process: **PSL and SVA**.
- **Functional languages** are those languages that we use to define **the function of a system**. These are thus the most commonly used languages of our industry. **Both imperative and declarative languages** can be used to define functionality.

# Models and Languages III

- The **dynamic languages and models** can be further subdivided into **algorithmic** approaches and **architectural** modeling approaches. The most prominent and widely used **architectural modeling language** is **SystemC**.
- **Algorithmic modeling languages** can be subdivided into **mathematical** modeling textual languages, the most famous being MATLAB and **dataflow** modeling languages, like Simulink.
- **Non-functional** languages and models are intended to capture characteristics of a system such as **constraints** (e.g., on energy consumption, area, cost) that define boundaries on the allowable or feasible implementations of those systems - **SDC**.
- **Meta-models** are usually pragmatically defined aggregates of information about a system or its components that are “**left out**” of all the other models in use - SPIRIT's **IP-XACT**.

# Language Problem in ESL

- **ESL spans two domains**, namely the hardware and software domains. These two domain use **different computation models** and that is the problem.
- With the migration to ESL, the **discrete event semantic is too limiting**.
- ESL needs the language which potential can **model both domains** equally good. That language is **SystemC**.

- SystemC is based on a single fixed model of computation, the same as the existing HDLs: namely, it follows the **discrete event semantics**. But because SystemC is built on an extensible framework, namely C++ it is much **more flexible**.
- In addition, given that it is based on C++, which itself is an extension of C, it **fully supports the legacy software** that exists.
- It has been constructed to **support multiple levels of abstraction** within the discrete event domain ranging from RTL constructs up to transaction modeling.
- It can be **extended** to support additional communications capabilities
  - **TLM 2.0**